

# Observability, Logging and the Log Server

How every process in the trading system reports what it is doing -  
and how you read it back when something goes wrong.

pm-log-srv

pm-log-cli

pm-log-ui

LALF

# 01

---

## Why observability matters

A trading system is many processes at once. Observability is how you see inside them.



# Observability is being able to answer questions you didn't plan for

Monitoring tells you that something broke. Observability lets you work out why — from evidence the system recorded before anyone knew there was a problem.

## WHAT

### What happened

The literal record: which process, which subsystem, which line of code, at what moment.

## WHEN

### In what order

Sequence is the whole game. A cause always precedes its effect — but only if the clocks agree.

## WHY

### Under what conditions

The surrounding lines: the warning ten seconds earlier, the retry loop, the traceback.

*In EduMatcher this is not a luxury. The engine, the gateways, the bots and the admin tools all run as separate processes — nobody can watch every terminal at once.*

# The same incident, with and without logging

Nothing about the failure changes. What changes is whether you can see it.

## WITHOUT CENTRAL LOGGING

### **“It stopped working.”**

- Four terminals, four scrollbacks
- Two of them already closed
- Timestamps in different formats
- No idea which process failed first
- Evidence disappears on restart

## WITH PM-LOG-SRV RUNNING

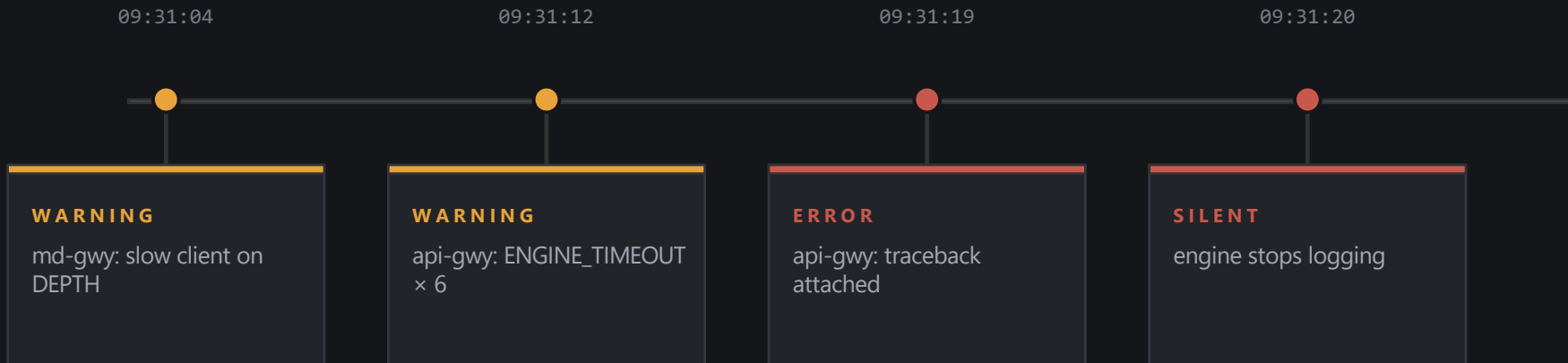
### **“Here is the first error.”**

- One database, every process
- One monotonic ordering key
- Full tracebacks preserved
- Query by level, process, time
- Still there tomorrow morning

*Logging is the cheapest insurance in the system. You pay for it before the incident, never during.*

# Post-mortem: reconstructing a lost minute

A post-mortem is archaeology. You cannot re-run the market; you can only read what the system left behind.

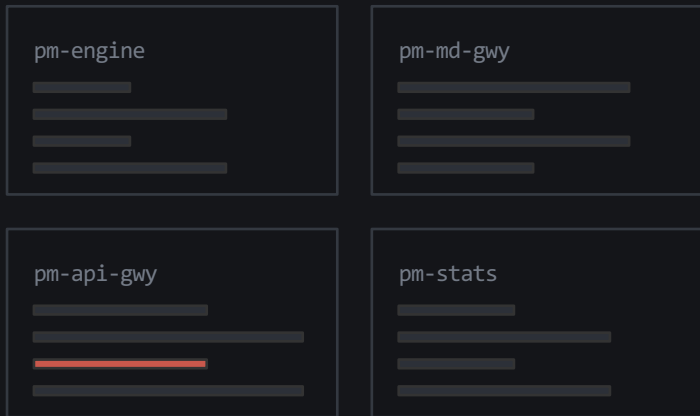


The conclusion writes itself once the rows sit in one table: **the gateway did not cause the outage — it was the first thing to notice it.**

# The problem a central collector actually solves

Every process already logs. The difficulty was never producing logs — it was reading four of them at the same time.

## BEFORE — FOUR SCROLLBACKS



LALF

## AFTER — ONE QUERYABLE TABLE

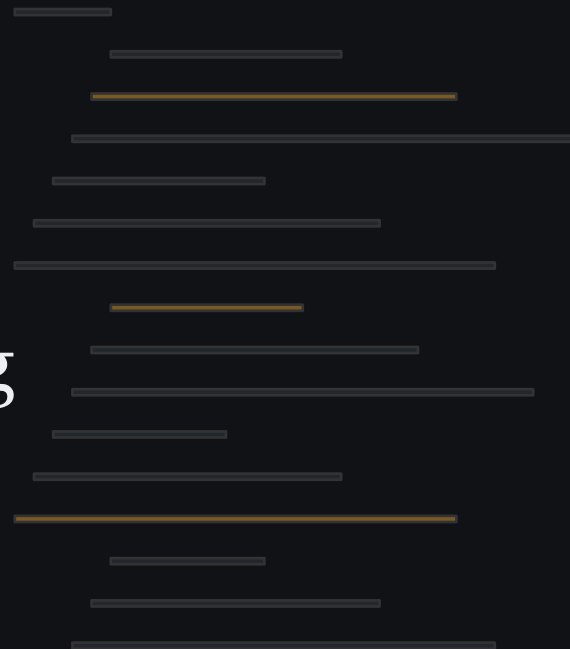
| log.db   |      |          |
|----------|------|----------|
| 09:31:04 | WARN | md-gwy   |
| 09:31:12 | WARN | api-gwy  |
| 09:31:19 | ERR  | api-gwy  |
| 09:31:21 | INFO | pm-stats |
| 09:31:24 | ERR  | engine   |

*Same lines. Same processes. One ordering. That single change is what makes an incident reviewable at all.*

# 02

## Three ways to log

Terminal, file, and the central log server — and when each one is the right answer.



# Every process can log to three different places

These are not alternatives you must choose between once. Most operators run all three at the same time, for different reasons.

01

## Terminal

`stdout / stderr`

Immediate, human-readable, and gone as soon as you close the window or the scrollback fills.

While you are watching

02

## File

`redirected output`

Survives the session and needs nothing running. One file per process, so cross-process work is manual.

Unattended runs

03

## Log server

`LALF → pm-log-srv`

One long-lived TCP connection per process into one SQLite database that every tool can query.

Anything you may investigate later

*pm-log-srv is the one process that never uses the log server — it always writes to stdout or a file, because nothing is listening for its own startup messages yet.*



# Terminal and file logging: what they are good for

The central server does not retire these. It sits alongside them.

## TERMINAL

The first thing you look at when starting a process. If a gateway fails to bind its port, you see it here before any collector could have stored it.

## FILE

The dependable fallback. If pm-log-srv is down, or the process cannot reach it, output still lands somewhere durable on disk.

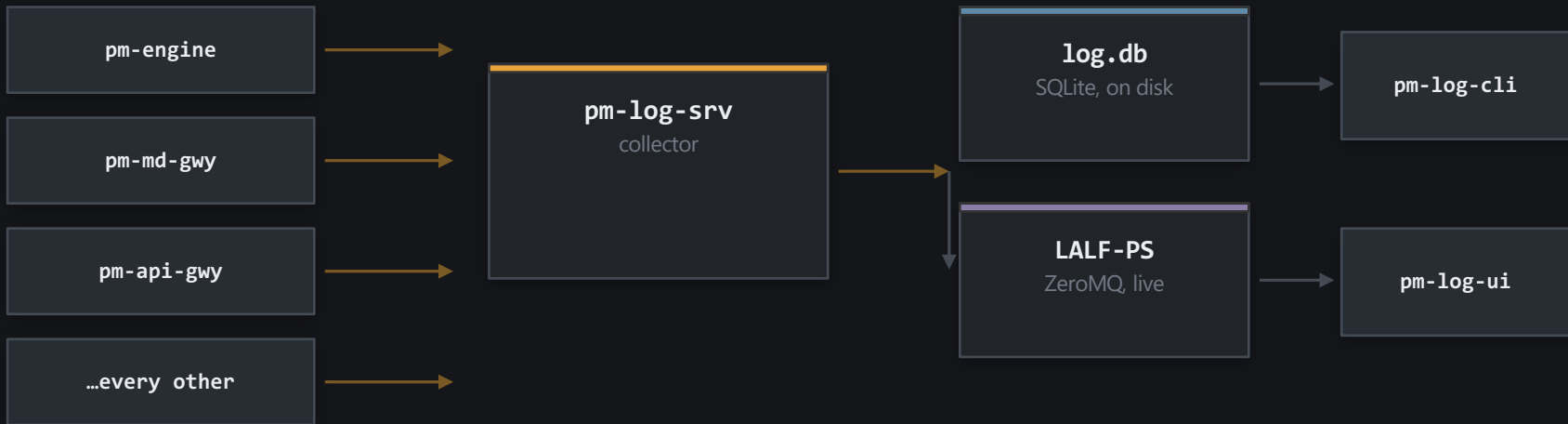
## THE TWO LOCAL SINKS

|                                                                  |                                                        |
|------------------------------------------------------------------|--------------------------------------------------------|
| <code>\$ pm-md-gwy</code>                                        | <code># terminal only – you are watching it</code>     |
| <code>\$ pm-md-gwy &gt; logs/md-gwy.log 2&gt;&amp;1 &amp;</code> | <code># file – unattended, survives the session</code> |
| <code>\$ tail -f logs/md-gwy.log</code>                          | <code># follow one file, one process</code>            |

*Notice the limit in that last line: it follows one process. Correlating two of them is you, with two windows and a clock.*

# What the central log server adds on top

pm-log-srv is a dedicated collector process. Every pm-\* process opens one connection to it and streams its own log records over the LALF protocol.

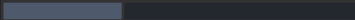
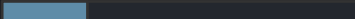
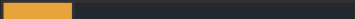
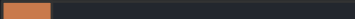
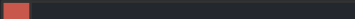


LALF / TCP :5600

*Three consequences worth remembering: every process is in one ordering, the data outlives the processes, and the tools that read it are completely separate from the one that writes it.*

# Log levels: the dial that decides how much you see

Every record carries exactly one of five levels. Filtering by level is the single most effective thing you can do when a system is noisy.

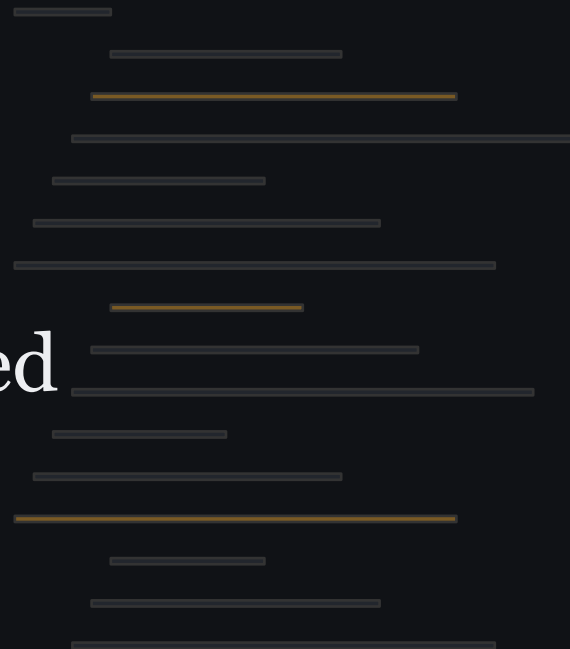
|          |                                                                                                |                                                                                     |
|----------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| DEBUG    | Internal detail for developers. Enormous volume — turn on deliberately, turn off again.        |  |
| INFO     | Normal milestones: started, connected, listening. This is the healthy heartbeat of the system. |  |
| WARNING  | Something is off but the process continues. This is where an incident usually starts.          |  |
| ERROR    | An operation failed. Often carries a traceback you can read directly.                          |  |
| CRITICAL | The process cannot continue doing its job. Always investigate.                                 |  |

Bar length shows typical relative volume: *DEBUG* dominates, *CRITICAL* is rare. Filtering to *WARNING* and above is the usual starting point on a busy day.

# 03

## How logs are stored

One SQLite file, one row per record, one monotonic sequence number.



# log.db — one SQLite file, not a pile of rotated text

SQLite was chosen for a reason: it is a single ordinary file that any tool can open read-only, with no server, no daemon, and no format to parse yourself.

**1**

single file on disk —  
copy it, archive it, mail it

**SQL**

any tool that speaks SQL  
can read it, live

**R/O**

readers never block or  
corrupt the collector

**seq**

server-assigned  
monotonic ordering key

## WHAT ONE STORED ROW CARRIES

|     |           |         |          |     |      |         |       |        |        |      |         |
|-----|-----------|---------|----------|-----|------|---------|-------|--------|--------|------|---------|
| seq | client_ts | process | instance | pid | host | session | level | logger | module | line | message |
| exc | truncated |         |          |     |      |         |       |        |        |      |         |

*Two tables in practice: log\_events holds the records; a connection registry records which process connected when, from where, and how much it has logged.*

# Anatomy of a single log record

This is what a process sends, and what you get back when you query. Learn these field names once — every tool in the chain uses them.

## ONE RECORD ON THE WIRE

```
LOG|SEQ=1042|TS=2026-07-28T14:32:07.511Z|LEVEL=WARNING|LOGGER=edumatcher.md_gateway.gateway|LEN=57  
slow client detected on channel DEPTH, symbol AAPL, dropping
```

### SEQ

Position in this connection's stream. Restarts at 1 on reconnect.

### TS

UTC timestamp with milliseconds, set by the sending process.

### LEVEL

One of the five severities. Anything else is rejected.

### LOGGER

Dotted name of the subsystem — your best filter after level.

### MODULE / LINE

Where in the source the record was emitted, when available.

### LEN

Exact byte count of the message that follows. Never guessed.

*The message body is length-prefixed, not line-delimited — which is precisely why a full multi-line traceback survives the trip intact.*

# Where the database lives, and how long rows stay

Two settings every operator should be able to state from memory: the path, and the retention window.

## DEFAULT LOCATION

`<repo>/src/data/log.db`

when run from a source checkout

`~/local/share/edumatcher/log.db`

when installed on your PATH

## OVERRIDING IT — AND WHY YOU WOULD

Point a whole session at its own database so a training run, a test, or a replay never mixes into your real operational history.

`EDUMATCHER_DATA_DIR=~/sessions/morning`

`pm-log-srv --db /tmp/session.db`

## RETENTION — 30 DAYS BY DEFAULT, PRUNED HOURLY

kept and queryable

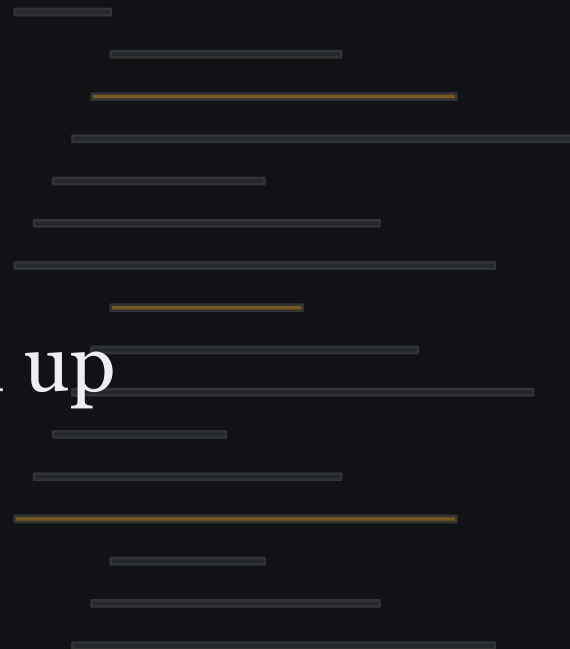
older than 30 days — deleted

*Operational logging is deliberately disposable — unlike the audit trail and market statistics, which are kept forever. Use `--retention-days 0` to disable pruning entirely, or `--retention-days 7` to keep the file small.*

# 04

## Bringing the system up

Starting the collector, the ports it claims, and the order that makes everything else easy.





# Starting the log server

With no arguments it reads engine\_config.yaml, binds its ports and starts writing. That is genuinely all it takes on a normal day.

## COMMON INVOCATIONS

```
$ pm-log-srv
```

```
$ pm-log-srv --host 127.0.0.1 \  
    --port 5600 -v
```

```
$ pm-log-srv --retention-days 0
```

```
$ pm-log-srv --db /tmp/session.db
```

## WHAT A HEALTHY START LOOKS LIKE

```
INFO main - starting pm-log-srv  
           with log level INFO  
  
INFO server - pm-log-srv 'log-srv01'  
           listening on 127.0.0.1:5600  
           db=data/log.db  
           retention_days=30  
  
INFO pubsub - LALF-PS interface up:  
           PUB=:5601  PULL=:5602  lease=30s
```

### Read those three lines before you start anything else

They tell you the bind address clients must reach, the exact database file every query tool must open, and whether live distribution came up. Almost every later confusion traces back to one of these three not being what you assumed.

# One process, three ports - and they are not interchangeable

pm-log-srv occupies a contiguous block. All three must be different or the server refuses to start.

## 5600

### LALF / TCP

Logs coming in. Every pm-\* process connects here to send its own records.

--port

**Producers**

## 5601

### LALF-PS PUB

Live rows going out over ZeroMQ, pushed the moment each row is committed.

--pub-port

**Live viewers**

## 5602

### LALF-PS PULL

Subscriber control coming in: subscribe, renew, unsubscribe, status.

--pull-port

**Live viewers**

Running two log servers on one host? **pm-log-srv --port 5700 --pub-port 5701 --pull-port 5702** — move all three together, never just one.

# The flags worth knowing - and why each one exists

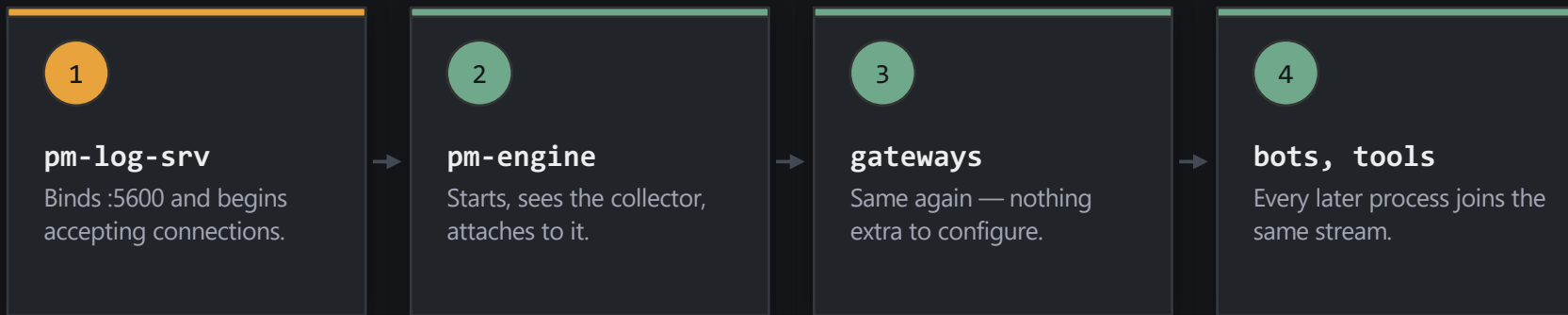
You can run the server with none of these. You reach for them when your situation stops being the default one.

| FLAG                               | DEFAULT            | WHEN YOU WOULD USE IT                                                                               |
|------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------|
| <code>--host ADDR</code>           | 0.0.0.0            | Use 127.0.0.1 when everything runs on one machine — it stops anything off-box from connecting.      |
| <code>--port PORT</code>           | 5600               | Move the collector port when something else already holds 5600, or you run a second server.         |
| <code>--db PATH</code>             | data/log.db        | Send a scratch or training session to its own file so it never mixes with real history.             |
| <code>--retention-days N</code>    | 30                 | 0 disables pruning entirely — use it when you must keep everything for a review.                    |
| <code>--max-message-bytes N</code> | 65536              | Raise it if long tracebacks are being truncated. Oversized records are cut, never dropped.          |
| <code>--no-pubsub</code>           | off                | Bind no ZeroMQ sockets at all. Pure collector, no live viewers — the console will not work.         |
| <code>--log-level LEVEL</code>     | WARNING            | The server's own verbosity. Use <code>-v</code> while setting things up so you see what it decided. |
| <code>--config PATH</code>         | engine_config.yaml | Where the optional <code>log_server: block</code> is read from. Flags always win over the file.     |

*A command-line flag overrides the config file for that run only — nothing on disk changes.*

# Start the log server first. Everything else gets easier.

Processes look for a running collector as they start. If one is already listening, they attach to it automatically — with no configuration on your part.



## COLLECTOR UP FIRST

Every process is in the database from its very first line — including the startup messages that explain a bad configuration.

## COLLECTOR UP LATE

Processes that already started keep logging locally. Their first minutes exist only in a terminal you may have closed.

# Nothing breaks if the collector is missing - you just lose the evidence<sup>21</sup>

This is a deliberate design choice. Logging must never be able to take the trading system down with it.

## No collector at startup

The process falls back to its terminal or file sink and carries on trading normally. Nothing is retried in the background.

Falls back

## Collector stops mid-session

The connection closes. Rows already accepted are flushed and stored; records generated after that were never buffered anywhere.

Gap in history

## Collector comes back

A reconnecting process starts a completely fresh session. There is no replay — the missing window stays missing.

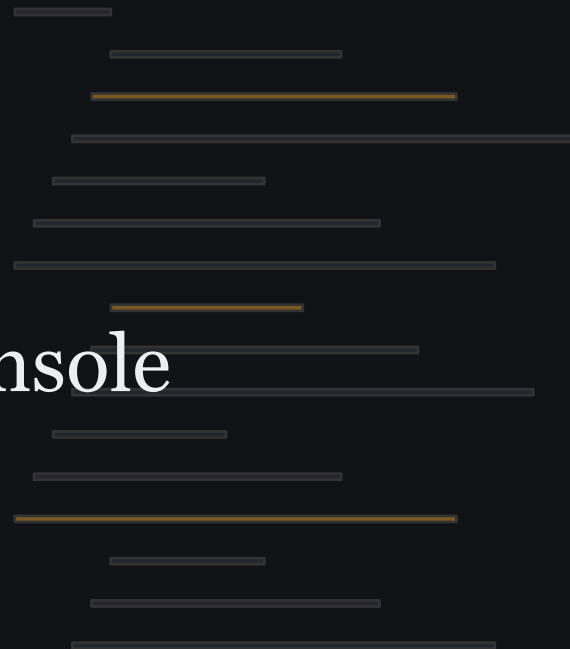
No replay

*Restated as one habit: bring pm-log-srv up before the engine, and leave it up for as long as the system is running.*

# 05

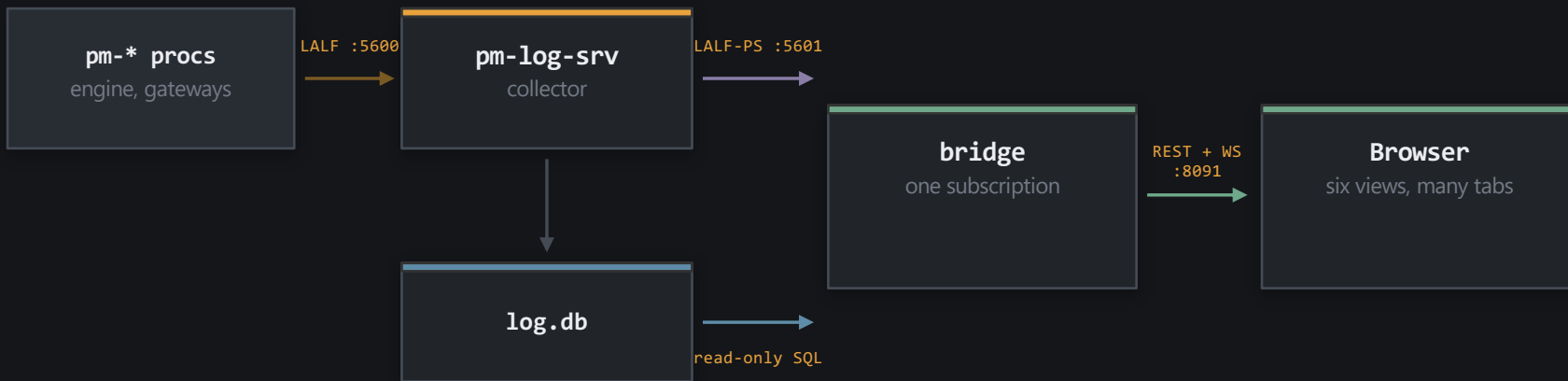
## The Log Operator Console

A browser application over the same data — built to answer “is anything wrong right now, and has someone dealt with it?”



# Where the console sits in the chain

The console is strictly additive. It never writes to log.db, never changes how collection behaves, and can be stopped at any moment without affecting anything.



**One bridge, however many operators.** Ten people watching the console cost the log server exactly one subscriber — the bridge fans out to browser tabs over its own connection.

# Two sources that fail independently - and are reported separately

The top bar shows two distinct red states rather than one “disconnected” light, because they mean different things and have different fixes.

## Live channel

LALF-PS from pm-log-srv

New rows as they land, plus server heartbeats

✗ Stops when pm-log-srv stops

✓ Keeps working if log.db moves

Pushed on commit

## History channel

log.db read from disk

Search, aggregation, counts, export

✗ Stops if the file is unreachable

✓ Keeps working when the server is stopped

Query time

**Why this matters to you:** stop the log server and the console still searches, aggregates and exports everything it already collected. The banner reads **log server down**, the live tail freezes, and nothing else is affected.



# Starting the console - three routes

In increasing order of setup effort. Almost everyone wants the first one.

01

## Container stack

Needs Podman or Docker, plus Make

```
$ cd log-gui  
$ make up
```

Auto-detects the runtime, builds the image and starts the stack in the background.

<http://localhost:8091>

02

## Pre-built image

Needs Podman or Docker only

```
$ make dist  
$ podman load --input ...tar.gz
```

A self-contained archive for a host with no repository checkout and no Node installed.

<http://localhost:8091>

03

## Local development

Needs Node.js and npm

```
$ make install  
$ make dev
```

Runs the bridge and a hot-reloading dev server together. For working on the console itself.

<http://127.0.0.1:5178>

*Whichever route you take, the console must be able to reach both of its sources: ports 5601 and 5602 on the log server, and a filesystem path to log.db. It has no use at all for port 5600.*

# The configuration you actually have to get right

Everything is an environment variable with a sensible default. These four are the ones that cause real confusion when they are wrong.

## LOG\_SRV\_HOST

default: 127.0.0.1

Where pm-log-srv is reachable from the bridge. Inside a container, "localhost" means the container itself — use host.docker.internal, host.containers.internal, or a real hostname.

## LOG\_DB\_PATH

default: resolved data dir /log.db

The collector's database, opened read-only. A bridge pointed at a different but valid database looks exactly like a working one with nothing in it.

## ACK\_STORE\_PATH

default: resolved data dir /log-ui-acks.db

The console's own store — the only thing it writes. Put it on a persistent volume or your acknowledgements vanish on restart.

## CORS\_ORIGIN

default: \*

Restrict this the moment the console is reachable beyond localhost. There is no login: anyone who reaches the port reads every log line.

*One more worth knowing: the Diagnostics view shells out to pm-log-cli, so that tool must be on PATH where the bridge runs — which, in the default container, it is not. Point LOG\_CLI\_COMMAND at it, or accept that one view reporting unavailable while everything else works.*

# The console at a glance

A fixed shell: a top bar carrying global state, a six-item navigation rail on the left, and the active view filling the rest.

drop the image here

## SCREENSHOT — MAIN WINDOW

Dashboard in dark theme with at least one unacknowledged issue, so the red banner, the top-bar unacked link and the Alerts rail badge are all visible together.

### Source state dot

Green when the log server is up, yellow while reconnecting, red for either failure.

### Unacked count

A link straight to Alerts, styled red whenever it is non-zero.

### Theme toggle

The icon shows the theme a click switches to, not the one you are in.

# The Dashboard, readable from across the room

Five meter tiles, two charts and the five most recent errors — designed so a passing glance is enough to tell whether anything needs you.

drop the image here

## SCREENSHOT — DASHBOARD WITH AN ACTIVE ALERT

The full Dashboard during a period with errors: the pulsing banner, all five meter tiles with real numbers, both charts populated, and the recent-errors list.

### Alert banner

Shown only when something is unacknowledged. No red at all is itself the signal.

### Five meters

Total events, error rate per minute, warnings in 60 s, process count, server state.

### Two charts

Events per minute over the last hour, and the processes producing the most errors.

# Six views, six different questions

Each view exists to answer one question well. Knowing which one to open is most of the skill.



## Dashboard

Is anything wrong right now?

refresh · Live + 15–30 s



## Logs

What exactly happened?

refresh · Live tail or on demand



## Alerts

What is outstanding, and who has it?

refresh · 10 s



## Processes

Is everything still reporting in?

refresh · 10 s



## Diagnostics

What would the diagnose rules say?

refresh · 60 s



## Health

Which component is degraded?

refresh · 10–30 s

*The rule of thumb: Dashboard to notice, Logs to investigate, Alerts to coordinate, Health to explain why a view is empty.*

# Reading the error-rate meter

The tile shows a word and a colour, not just a number — because “18 errors a minute” means nothing until you know what this venue considers busy.

| NORMAL                           | ELEVATED                         | HIGH                                           | SEVERE                                       |
|----------------------------------|----------------------------------|------------------------------------------------|----------------------------------------------|
| under 5 / min                    | 5 – 19 / min                     | 20 – 99 / min                                  | 100+ / min                                   |
| Background noise. Nothing to do. | Worth a glance at the Logs view. | Something is genuinely wrong. Investigate now. | A component is failing repeatedly. Escalate. |

## THE THRESHOLDS ARE YOURS TO SET

```
ERROR_RATE_NORMAL_PER_MIN=5
ERROR_RATE_ELEVATED_PER_MIN=20
ERROR_RATE_SEVERE_PER_MIN=100
```

## WHY TILES BRIEFLY DISAGREE

The error rate and the 60-second warning count come from the live stream and update every second. Everything else is polled from the database every 15–30 seconds. During a burst the live counters are simply ahead.

# The Logs explorer: live tail and historical search

One table, two modes, one filter bar that applies to both. This is where you spend most of an investigation.

## LIVE

Streams from the bridge into an in-memory buffer of the most recent 2 000 rows, newest first. Rows flash briefly as they arrive.

Scrolling down pauses the tail on purpose — you cannot read a log that keeps jumping. A banner counts what arrived while paused; click it to resume.

## HISTORY

Queries the database through the bridge and shows a match count, so you know whether your filter is finding anything at all.

Capped at 5 000 rows per query. Fewer rows than expected usually means the cap, not a gap — narrow the filter or export instead.

message contains...

level: WARNING+

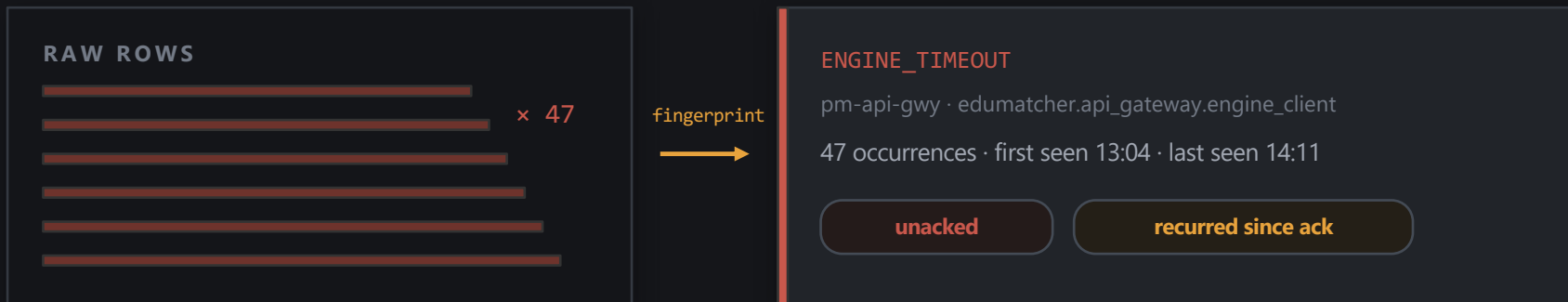
exceptions only

process = pm-md-gwy

**Every filter lives in the URL.** A filtered view is therefore a link — paste it into a ticket or a chat and a colleague opens exactly what you were looking at. Clicking any row opens a detail pane with the full field set and the untruncated message.

# Alerts: where the console earns its keep

Warnings and above are fingerprinted — the same fault recurring a thousand times collapses into one entry with a count, instead of a thousand lines to scroll past.



## Acknowledge

Marks it handled, attributed to your name, with an optional note recording what was done.

## Recurred since ack

Flagged prominently when a fingerprint fires again after being acknowledged — the fix did not hold.

## Shared, not personal

Acks live in the bridge's own store, so every operator sees the same state. It is a statement to your colleagues.



# Processes, Diagnostics and Health

The three views you open when something looks wrong but you do not yet know what.

## Processes

Every process that has ever connected, with a status glyph.

- Connected and logging
- ⚠ Connected but silent 30 s+
- Disconnected

The ⚠ state is the one to watch: a process that connected then went quiet shows nothing at all in the logs, which is precisely the problem.

## Diagnostics

Runs the same rule-based heuristics the command-line tool uses, every 60 seconds.

```
[WARNING] repeated_warning
pm-api-gwy logged "ENGINE_TIMEOUT"
47 times in the last hour
```

```
reproduce with: pm-log-cli query ...
```

Every finding cites the exact command that reproduces it — so you can always drill from “this is flagged” to the actual log lines.

## Health

Four panels for four components that fail independently — kept separate, never merged into one light.

|            |                                  |
|------------|----------------------------------|
| pm-log-srv | events, connections, truncations |
|------------|----------------------------------|

|         |                                   |
|---------|-----------------------------------|
| LALF-PS | subscribers, last sequence, drops |
|---------|-----------------------------------|

|        |                              |
|--------|------------------------------|
| log.db | path, size, rows, oldest row |
|--------|------------------------------|

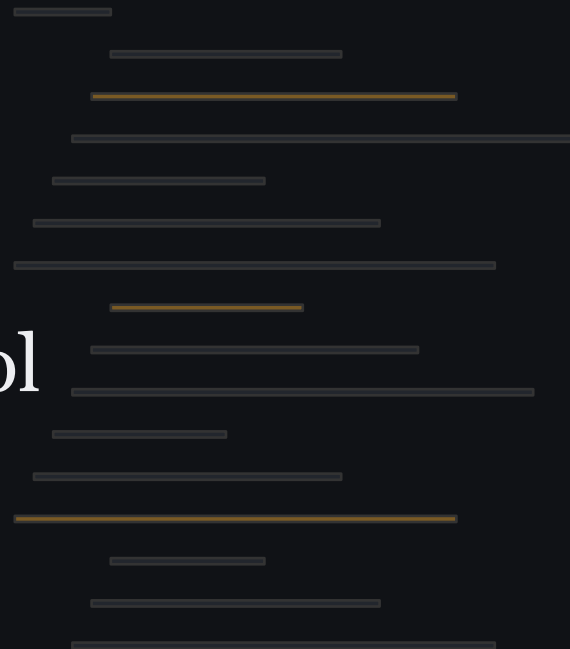
|        |                              |
|--------|------------------------------|
| Bridge | clients, acks stored, sub id |
|--------|------------------------------|

Start here. The resolved log.db path alone answers the most common misconfiguration of all.

# 06

## The LALF protocol

A deliberately small wire contract — and the reasons every one of its rules is there.



# Why a purpose-built protocol at all?

LALF stands for Logging ALF. It is the sibling of the protocols the trading system already speaks — same shape, one structural difference.

|                         |                            |                     |                    |                         |
|-------------------------|----------------------------|---------------------|--------------------|-------------------------|
| ALF<br>Text order entry | BALF<br>Binary order entry | CALF<br>Market data | RALF<br>Post-trade | LALF<br>Process logging |
|-------------------------|----------------------------|---------------------|--------------------|-------------------------|

## A log line is arbitrary text

It can contain pipes, newlines, and any byte at all — a traceback certainly does. So the message body is length-prefixed, and read by exact byte count. Nothing is ever scanned for a delimiter.

## Every client is only a producer

A process can send its own records and nothing else. There is no subscription model, no filtering, no way to read anyone else's stream. That is what makes it safe to wire into every process.

## Fire and forget, by design

The server never acknowledges individual records. At realistic logging rates, per-message confirmation would be pure overhead — and a process must never be slowed down by its own logging.

# One session, from HELLO to EXIT

A long-lived TCP connection, one handshake, then a stream of records. This is the whole life of a client.

## A COMPLETE SESSION

C: HELLO|CLIENT=pm-md-gwy|PID=51002|HOST=trader-laptop|PROTO=LALF1

S: WELCOME|PROTO=LALF1|SRV=log-srv01|HBINT=5|SESSION=7f3a9c21

C: LOG|SEQ=1|TS=...09:30:00.010Z|LEVEL=INFO|LOGGER=...|LEN=29

md-gwy01 listening on :5570

C: HB|TS=2026-07-28T09:30:05.500Z

C: EXIT

## TWO RULES THAT TRIP PEOPLE UP

SEQ counts per connection, not globally — it restarts at 1 every time a process reconnects, and the session id is what tells two connections apart.

An error reply is usually not fatal. Only a protocol mismatch or a missed handshake ends the session.

### HELLO

Identifies the process. Must arrive within 5 seconds of connecting or the server closes the socket.

### WELCOME

The only positive acknowledgement of the handshake. Carries the heartbeat interval and a session id.

### LOG

The only message that carries a payload, and the only one durably stored.

### HB

Sent at least every HBINT seconds even when quiet. Silence for twice that marks the connection dead.

### EXIT

A clean goodbye. Buffered rows are flushed first. A crash is treated identically.

# LALF brings logs in. LALF-PS sends them back out.

This is the single most common source of confusion. They are two separate interfaces on the same process, and nothing speaks both.

## LALF

process → log server

TCP :5600, line-oriented text

Unidirectional and subscription-free: a client can only ever send its own records, never read anyone else's.

**Every process that logs**

## LALF-PS

log server → subscriber

ZeroMQ PUB :5601 / PULL :5602, JSON

A viewer needs to read everything — the exact opposite of what LALF offers. So the server exposes a second interface.

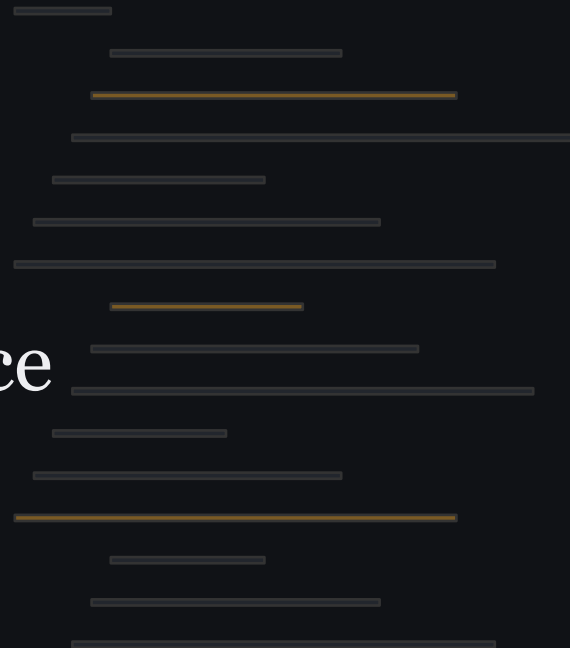
**Live viewers, including the console**

Practical consequence: the console never connects to port 5600, and no logging process ever connects to 5601 or 5602. If you are debugging connectivity, check the right pair of ports for the right side.

# 07

## Technical reference

The tables to come back to — message types, fields, codes, commands and settings.



# LALF message catalog

---

Nine message types. Only one of them carries a payload.

| MESSAGE | DIRECTION        | PURPOSE                                                                       |
|---------|------------------|-------------------------------------------------------------------------------|
| HELLO   | client → server  | Starts the session and identifies the connecting process. Must be first.      |
| WELCOME | server → client  | Confirms the session; carries server name, heartbeat interval and session id. |
| LOG     | client → server  | One formatted log record: header fields plus a length-prefixed body.          |
| ACK     | server → client  | Reserved for a future revision. The shipped server never sends it.            |
| ERR     | server → client  | A protocol violation or an advisory condition on something the client sent.   |
| HB      | client → server  | Heartbeat, sent whether or not anything was logged recently.                  |
| PING    | either direction | On-demand liveness check. In practice only a client initiates one.            |
| PONG    | either direction | Reply to PING. Carries no fields.                                             |
| EXIT    | client → server  | Clean disconnect. Buffered rows are flushed before the socket closes.         |

# LOG header fields

The field set is fixed in this revision. Five are required; the rest are supplied when available.

| FIELD  | REQ | TYPE   | DESCRIPTION                                                              |
|--------|-----|--------|--------------------------------------------------------------------------|
| SEQ    | yes | int    | Monotonic per-connection sequence number, starting at 1.                 |
| TS     | yes | string | UTC ISO-8601 timestamp with milliseconds, set by the client at emission. |
| LEVEL  | yes | enum   | DEBUG, INFO, WARNING, ERROR or CRITICAL. Anything else is rejected.      |
| LOGGER | yes | string | Originating logger name, e.g. edumatcher.md_gateway.gateway.             |
| MODULE | no  | string | Source module the record came from, when available.                      |
| LINE   | no  | int    | Source line number, when available.                                      |
| EXC    | no  | bool   | Set when the payload includes a formatted exception or traceback.        |
| LEN    | yes | int    | Exact byte length of the UTF-8 payload following the header line.        |

*Oversized payloads are truncated to `max_message_bytes` and stored with a truncated flag — never dropped, and never split mid-character.*



# Error codes — and which ones end the session

Most errors are not fatal. A client that disconnects on every error will drop a connection over one malformed line.

| CODE              | MEANING                                        | ENDS SESSION?                                        |
|-------------------|------------------------------------------------|------------------------------------------------------|
| INVALID_LEVEL     | The level was not one of the five valid values | No — fix the sending code and carry on               |
| MISSING_FIELD     | A required header field was absent             | No — same as above                                   |
| PAYLOAD_TOO_LARGE | The record exceeded max_message_bytes          | No — advisory only; already truncated and stored     |
| PROTO_MISMATCH    | Wrong protocol version, or a second handshake  | Yes — the server closes the connection               |
| HELLO_TIMEOUT     | No handshake within 5 seconds of connecting    | Yes — and you will probably never see it on the wire |

**On that last row:** the handshake-timeout error is queued and the socket closed in the same step, with no guarantee it is flushed first. Treat “connected, no welcome within a few seconds, then the socket closes” as the practical signal rather than waiting for a specific code.

# Transport, session and limits

The numbers behind the protocol. Most are fixed in this revision; the configurable ones name their setting.

| PROPERTY           | VALUE                           |
|--------------------|---------------------------------|
| Transport          | TCP, one connection per process |
| Default port       | 5600                            |
| Encoding           | UTF-8 throughout                |
| Line delimiter     | A single LF ends every header   |
| Payload framing    | Explicit byte count (LEN)       |
| Max header line    | 4096 bytes                      |
| Handshake deadline | 5 seconds after accept          |
| Heartbeat          | HBINT seconds, default 5        |
| Dead connection    | Silence for $2 \times$ HBINT    |

| SETTING                                        | DEFAULT |
|------------------------------------------------|---------|
| <code>log_server.port</code>                   | 5600    |
| <code>log_server.heartbeat_interval_sec</code> | 5       |
| <code>log_server.max_message_bytes</code>      | 65536   |
| <code>log_server.max_client_queue</code>       | 10000   |
| <code>lease_sec</code>                         | 30      |
| <code>max_subscribers</code>                   | 32      |
| <code>notify_interval_ms</code>                | 250     |
| <code>backfill_chunk_rows</code>               | 500     |
| <code>max_backfill_minutes</code>              | 1440    |

*Backpressure has no message of its own. When a client's internal queue grows past its limit, the server simply stops reading from that socket and lets TCP slow the sender down.*

# pm-log-cli — the command-line access layer

Read-only, reads the database directly, and works even when the collector is stopped. This remains the tool of record for scripting and large exports.

| COMMAND   | WHAT IT DOES                          | TYPICAL USE                                                        |
|-----------|---------------------------------------|--------------------------------------------------------------------|
| query     | Filtered historical search            | <code>pm-log-cli query --level ERROR,CRITICAL --since 13:00</code> |
| tail      | Follows new rows by polling           | <code>pm-log-cli tail --process pm-md-gwy</code>                   |
| processes | Who has connected, and who still is   | <code>pm-log-cli processes --active</code>                         |
| stats     | Lifetime counters and database health | <code>pm-log-cli stats --format json</code>                        |
| diagnose  | Rule-based troubleshooting report     | <code>pm-log-cli diagnose --since 13:00</code>                     |
| prune     | Deletes rows older than N days        | <code>pm-log-cli prune --days 7</code>                             |

## FILTERS ON QUERY

```
--process --level --logger --since --until  
--grep --has-exception --limit --reverse
```

## EXIT CODES

```
0 success      1 database unreadable  
2 bad arguments 3 diagnose found something
```

# The seven diagnose heuristics

Every one is a plain SQL aggregate plus a fixed threshold — never a statistical model. “Why did it flag this?” always has a one-sentence answer.

|                                   |                                                                                         |
|-----------------------------------|-----------------------------------------------------------------------------------------|
| <b>Error-rate spike</b>           | Errors in the last five minutes far exceed the preceding hour's baseline for a process. |
| <b>Repeated identical warning</b> | The same process, logger and message repeat many times in the window.                   |
| <b>Process silence</b>            | A still-connected session has not logged or heartbeated recently.                       |
| <b>Clock skew</b>                 | A process's timestamps run consistently ahead of or behind the server's own clock.      |
| <b>Truncated-message rate</b>     | A process sent oversized records that had to be cut short.                              |
| <b>Exception clustering</b>       | Most tracebacks in the window come from one specific logger.                            |
| <b>Likely fallback to file</b>    | A process disconnected cleanly and never came back — distinct from one merely stalled.  |

*When nothing fires, diagnose says so explicitly — a clean run is visibly a clean run, not an empty screen.*

# LALF-PS: subscription modes, filters and leases

What a live viewer asks the log server for, and the three mechanisms that keep the server from accumulating state for viewers that died.

## STREAM

Full rows pushed as they are committed. What a live tail wants — the viewer never touches the database at all.

## NOTIFY

Only a small tick: how many rows matched and the highest sequence. For a UI that reads the database itself and just needs to know when to refresh.

| FILTER FIELD                 | MATCHES                           |
|------------------------------|-----------------------------------|
| <code>min_level</code>       | Rows at or above this severity    |
| <code>processes</code>       | Exact process-name match          |
| <code>loggers</code>         | Prefix match on the logger name   |
| <code>sessions</code>        | One connection of one process     |
| <code>contains</code>        | Case-insensitive text in the body |
| <code>exceptions_only</code> | Only rows carrying a traceback    |

## WHY SUBSCRIPTIONS ARE LEASES

A publish socket is blind — it never learns that a subscriber died, and publishing into the void succeeds silently. So every subscription carries a time-to-live that must be renewed, and anything that stops renewing is reaped: filter state gone, buffers discarded.

*All present filter fields are combined with AND. A malformed filter is rejected outright rather than quietly matching everything — a filter that stops filtering is how a viewer ends up flooded.*

# Console environment variables

All bridge configuration is environment variables. Invalid values fall back to the default rather than failing startup.

| VARIABLE          | DEFAULT                  |
|-------------------|--------------------------|
| HOST              | 127.0.0.1                |
| PORT              | 8091                     |
| CORS_ORIGIN       | *                        |
| LOG_SRV_HOST      | 127.0.0.1                |
| LOG_SRV_PUB_PORT  | 5601                     |
| LOG_SRV_PULL_PORT | 5602                     |
| LEASE_SEC         | 30                       |
| LOG_DB_PATH       | data dir /log.db         |
| ACK_STORE_PATH    | data dir /log-ui-acks.db |
| LOG_CLI_COMMAND   | pm-log-cli               |

| VARIABLE                     | DEFAULT |
|------------------------------|---------|
| ISSUES_RETENTION_DAYS        | 7       |
| ISSUES_MIN_LEVEL             | WARNING |
| ISSUES_ALERT_LEVEL           | ERROR   |
| PROCESS_SILENCE_SEC          | 30      |
| ERROR_RATE_NORMAL_PER_MIN    | 5       |
| ERROR_RATE_ELEVATED_PER_MIN  | 20      |
| ERROR_RATE_SEVERE_PER_MIN    | 100     |
| QUERY_MAX_ROWS               | 5000    |
| EXPORT_MAX_ROWS              | 1000000 |
| LIVE_BATCH_THRESHOLD_PER_SEC | 50      |

*The two level variables are independent and the order matters: the minimum level decides what is tracked at all, the alert level decides what is counted as demanding attention.*

# Troubleshooting, in one table

The symptoms you are most likely to meet in the first month, and what each one usually means.

| SYMPTOM                               | LIKELY CAUSE                             | WHAT TO DO                                                    |
|---------------------------------------|------------------------------------------|---------------------------------------------------------------|
| Server refuses to start               | Its three ports are not all distinct     | Give each of 5600/5601/5602 a different value                 |
| "log server down", history fine       | The collector is stopped or unreachable  | Start it; check the host and that 5601/5602 are reachable     |
| "log.db unavailable", tail fine       | The bridge cannot open the file          | Compare the path in Health with where the server writes       |
| Every view empty, no error            | Reading a different, valid database      | Check the resolved paths logged at startup                    |
| Live tail stopped after scrolling     | Expected — scrolling pauses the tail     | Click the "new rows — jump to top" banner                     |
| "Diagnostics unavailable"             | pm-log-cli is not on PATH for the bridge | Set LOG_CLI_COMMAND, or run the bridge where it is installed  |
| Acks vanish after a restart           | The ack store is on ephemeral storage    | Mount a persistent volume — it is the only state you own      |
| An export looks complete but is short | The export row cap was reached           | Check the truncation header; narrow the filter or use the CLI |

# Four habits worth forming

01

## Start the log server first

Before the engine, before the gateways. Everything that starts later joins on its own.

02

## Know your three numbers

The bind address, the database path, and the retention window. Read them off the startup lines.

03

## Filter before you scroll

Level first, then process, then text. A whole-system search at DEBUG helps nobody.

04

## Acknowledge with a note

An acknowledgement is a message to your colleagues, not a way to hide a red badge.

*Logging is only worth what it is worth on your worst day. Everything above is preparation for that day.*